💎 SYSTÈME DE CRÉDITS GABON 24/7 - VERSION PREMIUM

🎯 NOUVELLE STRATÉGIE DE PRICING

Principe Psychologique

```
Prix d'achat attractif (500 FCFA = 2000 crédits)
MAIS consommation rapide (6-8x le coût réel)
→ L'utilisateur doit recharger fréquemment
→ Revenus récurrents maximisés
```

💰 GRILLE TARIFAIRE OPTIMISÉE

📦 PACKS DE CRÉDITS

Pack	Prix FCFA	Crédits	Bonus	Total Crédits	FCFA/crédit
Découverte	500	2,000	-	2,000	0.25
Starter	1,000	4,000	+200	4,200	0.24
Standard	2,500	10,000	+750	10,750	0.23
Pro	5,000	20,000	+2,000	22,000	0.23
Business	10,000	40,000	+5,000	45,000	0.22
Enterprise	25,000	100,000	+15,000	115,000	0.22

💸 COÛT PAR FONCTIONNALITÉ (INTERACTIONS UTILISATEUR UNIQUEMENT)

📊 Pricing Détaillé - Ratio 6-8x

Fonctionnalité	Coût OpenAI	Ratio	Prix Crédits	Utilisations avec 2000 crédits
Business Plan Complet	4.6 FCFA	8x	**850 crédits**	**2-3 plans** ⚡
Chat Projet (message)	1.2 FCFA	7x	**200 crédits**	**10 messages**
Actu Plus	2.0 FCFA	7x	**320 crédits**	**6-7 analyses**
Analyse Opportunités	1.1 FCFA	7x	**180 crédits**	**11 analyses**
Formation Module	0.8 FCFA	8x	**150 crédits**	**13 modules**

|
|**Plan d'Action** |0.8 FCFA |8x |**150 crédits**|**13 plans** ⚡
|
|**Test Compétences** |0.6 FCFA |7x |**100 crédits**|**20 tests**
|
|**Framework Projet** |0.4 FCFA |8x |**75 crédits** |**26 frameworks**
|
|**Résumé Custom** |0.3 FCFA |6x |**45 crédits** |**44 résumés**
|
|**Génération Courrier** |0.3 FCFA |6x |**45 crédits** |**44 courriers**
|
|**Sondage Audio** |0.1 FCFA |8x |**20 crédits** |**100 sondages**
|

🎯 Exemples Concrets avec Pack 500 FCFA (2000 crédits)

Scénario Entrepreneur :

```
500 FCFA = 2,000 crédits
→ 2 Business Plans complets (1,700 crédits)
→ + 1 Analyse Opportunités (180 crédits)
→ Solde : 120 crédits
→ Doit recharger pour continuer !
```

Scénario Professionnel :

```
500 FCFA = 2,000 crédits
→ 13 Plans d'Action (1,950 crédits)
→ Solde : 50 crédits
→ Doit recharger !
```

Scénario Étudiant :

```
500 FCFA = 2,000 crédits
→ 13 Formations complètes (1,950 crédits)
→ Solde : 50 crédits
```

📊 TABLEAU DE CONSOMMATION RAPIDE

Avec 2,000 crédits (500 FCFA), vous pouvez :

Option A	Option B	Option C
2 Business Plans	**13 Plans d'Action**	**6 Actu Plus**
+ 1 Analyse	OU 13 Formations	+ 10 Messages Chat
		+ 1 Framework

** 💡 Message Marketing :** *"Pour 500 FCFA seulement, créez 2 business plans professionnels complets !"*

💳 PACKAGES DÉTAILLÉS

Pack Découverte - 500 FCFA

```javascript
{
  price: 500,
  credits: 2000,
  marketing: "Parfait pour tester la plateforme",
  examples: [
    "✅ 2 Business Plans complets",
    "✅ 13 Plans d'action détaillés",
    "✅ 6 analyses Actu Plus",
    "✅ 10 conversations IA",
    "⏰ Valable 1 mois"
  ]
}
```

Pack Starter - 1,000 FCFA ⭐ POPULAIRE

```javascript
{
  price: 1000,
  credits: 4200, // +5% bonus
  marketing: "Idéal pour entrepreneurs",
  examples: [
    "✅ 4 Business Plans complets",
    "✅ 28 Plans d'action",
    "✅ 13 analyses Actu Plus",
    "✅ 23 opportunités business",
    "✅ Support standard",
    "⏰ Valable 3 mois"
```

  ],
  popular: true
}
```

### Pack Standard - 2,500 FCFA

```javascript
{
  price: 2500,
  credits: 10750, // +7.5% bonus
  marketing: "Pour professionnels actifs",
  examples: [
    "✅ 12 Business Plans",
    "✅ 71 Plans d'action",
    "✅ 33 analyses Actu Plus",
    "✅ 59 opportunités business",
    "✅ 50+ conversations IA",
    "🎁 Support prioritaire",
    "⏰ Valable 6 mois"
  ]
}
```

Pack Pro - 5,000 FCFA

```javascript
{
  price: 5000,
  credits: 22000, // +10% bonus
  marketing: "Solution business complète",
  examples: [
    "✅ 25 Business Plans",
    "✅ 146 Plans d'action",
    "✅ 68 analyses Actu Plus",
    "✅ 122 opportunités business",
    "✅ 110 conversations IA illimitées",
    "🎁 Support VIP 24/7",
    "🎁 Accès beta features",
    "⏰ Valable 1 an"
  ]
}
```

🎨 INTERFACE UTILISATEUR
```

1️⃣ Widget Crédits avec Urgence

```javascript
// Affichage dynamique du solde
const CreditWidget = () => {
  const getUrgencyLevel = (credits) => {
    if (credits < 200) return {
      color: 'red',
      message: '⚠️ Crédits presque épuisés !',
      cta: 'Recharger maintenant'
    };
    if (credits < 500) return {
      color: 'orange',
      message: '⚡ Pensez à recharger bientôt',
      cta: 'Voir les offres'
    };
    return {
      color: 'green',
      message: '✅ Solde suffisant',
      cta: null
    };
  };

  return (
    <div className="credit-widget" style={{borderColor: urgency.color}}>
      <div className="balance">
        💎 {user.credits.toLocaleString()} crédits
      </div>
      <div className="message">{urgency.message}</div>
      {urgency.cta && (
        <button className="cta-recharge">{urgency.cta}</button>
      )}
    </div>
  );
};
```

2️⃣ Confirmation Avant Action (Affichage Proéminent)

```javascript
// Modal de confirmation
const ActionConfirmation = ({ action }) => {
  return (
    <div className="action-modal">
      <h3>🎯 {action.name}</h3>
```

```
      <div className="cost-display">
        <div className="cost-amount">
          {action.credits} crédits
        </div>
        <div className="cost-equivalent">
          ≈ {(action.credits * 0.25).toFixed(0)} FCFA
        </div>
      </div>

      <div className="balance-after">
        <span>Solde actuel:</span>
        <strong>{user.credits} crédits</strong>
      </div>
      <div className="balance-after">
        <span>Solde après:</span>
        <strong className={newBalance < 200 ? 'warning' : ''}>
          {newBalance} crédits
        </strong>
      </div>

      {newBalance < 200 && (
        <div className="warning-message">
          ⚠️ Attention : Il vous restera moins de 200 crédits
          <button onClick={buyCredits}>
            Recharger d'abord (+10% bonus)
          </button>
        </div>
      )}

      <div className="actions">
        <button className="confirm" onClick={proceed}>
          Confirmer
        </button>
        <button className="cancel" onClick={cancel}>
          Annuler
        </button>
      </div>
    </div>
  );
};
```

3️⃣ Boutons d'Action avec Coût Visible

```javascript
// Tous les boutons IA affichent le coût
<button
```

```
  className="ai-action-btn"
  onClick={generateBusinessPlan}
  disabled={user.credits < 850}
>
  <span className="action-icon">📊</span>
  <span className="action-name">Générer Business Plan</span>
  <span className="action-cost">
    <span className="cost-icon">💎</span>
    850 crédits
  </span>
</button>

// Style visuel
.ai-action-btn {
  display: flex;
  justify-content: space-between;
  align-items: center;
  padding: 15px 20px;
  background: linear-gradient(135deg, #667eea 0%, #764ba2 100%);
  border-radius: 12px;
  color: white;
}

.action-cost {
  background: rgba(255,255,255,0.2);
  padding: 5px 12px;
  border-radius: 20px;
  font-weight: bold;
}
```

📊 PROJECTIONS FINANCIÈRES RÉALISTES

Scénario 1 : 100 Utilisateurs Actifs/Mois

```
Hypothèses :
- 70% achètent Pack Découverte (500 FCFA)
- 20% achètent Pack Starter (1,000 FCFA)
- 10% achètent Pack Standard (2,500 FCFA)
- Moyenne 2 recharges/utilisateur/mois

Calculs :
- 70 × 500 × 2 = 70,000 FCFA
- 20 × 1,000 × 2 = 40,000 FCFA
```

- 10 × 2,500 × 2 = 50,000 FCFA

REVENUS MENSUELS : 160,000 FCFA
COÛT IA (calculé) : 23,000 FCFA
MARGE NETTE : 137,000 FCFA (86% de marge)
```

### Scénario 2 : 500 Utilisateurs Actifs/Mois

```
REVENUS MENSUELS : 800,000 FCFA
COÛT IA : 114,000 FCFA
MARGE NETTE : 686,000 FCFA (86% de marge)

ANNUEL : 8,232,000 FCFA de profit
```

Scénario 3 : 1,000 Utilisateurs (Objectif 6 mois)

```
REVENUS MENSUELS : 1,600,000 FCFA
COÛT IA : 228,000 FCFA
MARGE NETTE : 1,372,000 FCFA

ANNUEL : 16,464,000 FCFA de profit
```

🎁 STRATÉGIES DE RÉTENTION

1. Notifications Push Intelligentes

```javascript
// Quand crédits < 300
"⚠️ Plus que 300 crédits !
Rechargez maintenant et recevez +10% de bonus 🎁"

// 3 jours sans utilisation
"👋 On vous a manqué !
Revenez aujourd'hui et recevez 200 crédits gratuits"

// Après première utilisation
"🎉 Première action réussie !
Parrainez un ami et recevez 500 crédits chacun"
```
```

2. Programme de Fidélité Agressif

```javascript
const loyaltyTiers = {
  bronze: {
    spent: 0,
    bonus: '0% bonus',
    perks: []
  },
  silver: {
    spent: 5000, // FCFA dépensés
    bonus: '+5% crédits sur chaque recharge',
    perks: ['Support prioritaire']
  },
  gold: {
    spent: 15000,
    bonus: '+10% crédits sur chaque recharge',
    perks: ['Support VIP', 'Accès beta', '1 business plan gratuit/mois']
  },
  platinum: {
    spent: 50000,
    bonus: '+15% crédits sur chaque recharge',
    perks: ['Support 24/7', 'Toutes beta features', '3 business plans gratuits/
mois', 'Consultation 1-on-1']
  }
};
```

3. Offres Flash Irrésistibles

```javascript
// Tous les vendredis
"🔥 FLASH WEEKEND 🔥
Pack Starter à 800 FCFA au lieu de 1,000
+ 500 crédits bonus
⏰ Expire dans 48h"

// Fin de mois
"💰 MEGA PROMO FIN DE MOIS
Pack Standard à 2,000 FCFA (-20%)
+ 1,000 crédits bonus gratuits
⏰ Jusqu'au 30 à minuit"
```

🔧 BASE DE DONNÉES

```sql
-- Table principale des crédits
CREATE TABLE user_credits (
  user_id UUID PRIMARY KEY,
  balance INTEGER DEFAULT 0,
  lifetime_purchased INTEGER DEFAULT 0,
  lifetime_spent INTEGER DEFAULT 0,
  tier VARCHAR(20) DEFAULT 'bronze',
  last_purchase_at TIMESTAMP,
  created_at TIMESTAMP DEFAULT NOW(),
  updated_at TIMESTAMP DEFAULT NOW()
);

-- Table des transactions (audit trail)
CREATE TABLE credit_transactions (
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  user_id UUID REFERENCES users(id),
  type VARCHAR(20), -- 'purchase', 'usage', 'bonus', 'refund'
  amount INTEGER, -- négatif pour usage
  balance_before INTEGER,
  balance_after INTEGER,
  feature VARCHAR(100), -- 'business_plan', 'chat_message', etc.
  package_id VARCHAR(50), -- si purchase
  payment_method VARCHAR(50), -- 'airtel', 'moov', etc.
  metadata JSONB,
  created_at TIMESTAMP DEFAULT NOW()
);

-- Table des packages
CREATE TABLE credit_packages (
  id VARCHAR(50) PRIMARY KEY,
  name VARCHAR(100),
  price_fcfa INTEGER,
  base_credits INTEGER,
  bonus_credits INTEGER,
  total_credits INTEGER GENERATED ALWAYS AS (base_credits +
bonus_credits) STORED,
  active BOOLEAN DEFAULT true,
  sort_order INTEGER,
  metadata JSONB
);

-- Index pour performance
CREATE INDEX idx_transactions_user ON credit_transactions(user_id,
```

created_at DESC);
CREATE INDEX idx_transactions_feature ON credit_transactions(feature);
```

---

## 🎯 TARIFS FONCTIONNALITÉS (Rappel)

```javascript
// Configuration centralisée
const FEATURE_COSTS = {
  // Fonctions principales
  business_plan_full: 850,
  chat_message: 200,
  actu_plus: 320,
  opportunity_analysis: 180,
  training_module: 150,
  action_plan: 150,
  skills_test: 100,
  project_framework: 75,
  custom_summary: 45,
  letter_generation: 45,
  audio_survey: 20,

  // Futures fonctionnalités
  business_intelligence_report: 500,
  market_research: 400,
  competitive_analysis: 300
};

// Fonction de déduction
async function consumeCredits(userId, feature, metadata = {}) {
  const cost = FEATURE_COSTS[feature];

  // Vérifier solde
  const { balance } = await getUserCredits(userId);

  if (balance < cost) {
    throw new InsufficientCreditsError({
      required: cost,
      available: balance,
      missing: cost - balance
    });
  }

  // Déduire
  await createTransaction({

```
    userId,
    type: 'usage',
    amount: -cost,
    feature,
    metadata
  });

  return {
    success: true,
    creditsUsed: cost,
    newBalance: balance - cost
  };
}
```

📱 INTÉGRATION MOBILE MONEY

```javascript
// API de paiement
async function initiateCreditPurchase(userId, packageId, phoneNumber,
operator) {
  const package = await getPackage(packageId);

  // Initier paiement Mobile Money
  const payment = await mobileMoneyAPI.initiate({
    amount: package.price_fcfa,
    phone: phoneNumber,
    operator, // 'airtel', 'moov', 'gtmoney'
    reference: `CREDIT_${userId}_${Date.now()}`,
    callback: `${API_URL}/credits/payment-callback`
  });

  // Créer transaction en attente
  await createPendingTransaction({
    userId,
    packageId,
    paymentId: payment.id,
    status: 'pending'
  });

  return {
    paymentId: payment.id,
    status: 'pending',
    message: 'Composez *123# pour confirmer le paiement'
  };
```

```
}

// Webhook de confirmation
async function handlePaymentCallback(paymentId, status) {
  if (status === 'success') {
    const transaction = await getPendingTransaction(paymentId);
    const package = await getPackage(transaction.packageId);

    // Créditer l'utilisateur
    await addCredits(transaction.userId, package.total_credits);

    // Enregistrer la transaction
    await createTransaction({
      userId: transaction.userId,
      type: 'purchase',
      amount: package.total_credits,
      packageId: package.id,
      paymentMethod: transaction.operator
    });

    // Notification
    await sendNotification(transaction.userId, {
      title: '✅ Recharge réussie !',
      body: `${package.total_credits} crédits ajoutés à votre compte`
    });
  }
}
```

✅ RÉSUMÉ FINAL

🎯 Ce Que Vous Obtenez :

✅ Pack Découverte attractif : 500 FCFA = 2,000 crédits
✅ Consommation rapide : 2-3 business plans max
✅ Recharges fréquentes obligatoires
✅ Marge nette : 86%
✅ Revenus récurrents garantis

💰 Projection Réaliste (500 utilisateurs) :

```
Revenus mensuels : 800,000 FCFA
Coût IA : 114,000 FCFA
Profit net : 686,000 FCFA/mois
```

SOIT : 8,232,000 FCFA/an
```

### 🚀 Prochaines Étapes :

1. Valider les tarifs finaux
1. Créer les tables Supabase
1. Implémenter l'intégration Mobile Money
1. Designer l'interface d'achat
1. Configurer les notifications

# 🚀 CODE COMPLET - SYSTÈME DE CRÉDITS GABON 24/7

## 📁 STRUCTURE DU PROJET

```
gabon247-credits/
├── supabase/
│   ├── migrations/
│   │   └── 20241116_credits_system.sql
│   └── functions/
│   ├── consume-credits/
│   └── payment-webhook/
├── src/
│   ├── lib/
│   │   ├── credits.js
│   │   ├── momo-payment.js
│   │   └── constants.js
│   ├── components/
│   │   ├── CreditWidget.jsx
│   │   ├── CreditPurchase.jsx
│   │   ├── ActionConfirmation.jsx
│   │   └── CreditHistory.jsx
│   ├── pages/
│   │   ├── credits/
│   │   │   ├── buy.jsx
│   │   │   └── history.jsx
│   │   └── api/
│   │   └── credits/
│   │   ├── purchase.js
│   │   ├── consume.js
│   │   ├── balance.js
│   │   └── webhook.js
│   └── hooks/
│   └── useCredits.js
└── styles/
```
```

```
    └──── credits.css
```

1️⃣ MIGRATION SUPABASE

```sql
-- supabase/migrations/20241116_credits_system.sql

-- Extension pour UUID
CREATE EXTENSION IF NOT EXISTS "uuid-ossp";

-- =====================
-- TABLE: credit_packages
-- =====================
CREATE TABLE credit_packages (
  id VARCHAR(50) PRIMARY KEY,
  name VARCHAR(100) NOT NULL,
  price_fcfa INTEGER NOT NULL,
  base_credits INTEGER NOT NULL,
  bonus_credits INTEGER DEFAULT 0,
  total_credits INTEGER GENERATED ALWAYS AS (base_credits +
bonus_credits) STORED,
  popular BOOLEAN DEFAULT false,
  active BOOLEAN DEFAULT true,
  sort_order INTEGER DEFAULT 0,
  features JSONB DEFAULT '[]',
  metadata JSONB DEFAULT '{}',
  created_at TIMESTAMP DEFAULT NOW(),
  updated_at TIMESTAMP DEFAULT NOW()
);

-- Données initiales des packages
INSERT INTO credit_packages (id, name, price_fcfa, base_credits,
bonus_credits, popular, sort_order, features) VALUES
('decouverte', 'Pack Découverte', 500, 2000, 0, false, 1,
  '["2 Business Plans complets", "13 Plans d''action détaillés", "6 analyses Actu
Plus", "10 conversations IA", "Valable 1 mois"]'::jsonb),
('starter', 'Pack Starter', 1000, 4000, 200, true, 2,
  '["4 Business Plans complets", "28 Plans d''action", "13 analyses Actu Plus",
"23 opportunités business", "Support standard", "Valable 3 mois"]'::jsonb),
('standard', 'Pack Standard', 2500, 10000, 750, false, 3,
  '["12 Business Plans", "71 Plans d''action", "33 analyses Actu Plus", "59
opportunités business", "50+ conversations IA", "Support prioritaire", "Valable
6 mois"]'::jsonb),
('pro', 'Pack Pro', 5000, 20000, 2000, false, 4,
```

```
  '["25 Business Plans", "146 Plans d''action", "68 analyses Actu Plus", "122
opportunités business", "Support VIP 24/7", "Accès beta features", "Valable 1
an"]'::jsonb),
('business', 'Pack Business', 10000, 40000, 5000, false, 5,
  '["50+ Business Plans", "293 Plans d''action", "Support VIP 24/7", "Accès
anticipé features", "Gestion multi-utilisateurs", "Valable 1 an"]'::jsonb),
('enterprise', 'Pack Enterprise', 25000, 100000, 15000, false, 6,
  '["Usage illimité", "Support dédié", "API access", "Intégration custom",
"Formation équipe", "Valable 1 an"]'::jsonb);

-- ======================
-- TABLE: user_credits
-- ======================
CREATE TABLE user_credits (
  user_id UUID PRIMARY KEY REFERENCES auth.users(id) ON DELETE
CASCADE,
  balance INTEGER DEFAULT 0 CHECK (balance >= 0),
  lifetime_purchased INTEGER DEFAULT 0,
  lifetime_spent INTEGER DEFAULT 0,
  tier VARCHAR(20) DEFAULT 'bronze',
  last_purchase_at TIMESTAMP,
  last_usage_at TIMESTAMP,
  created_at TIMESTAMP DEFAULT NOW(),
  updated_at TIMESTAMP DEFAULT NOW()
);

-- ======================
-- TABLE: credit_transactions
-- ======================
CREATE TABLE credit_transactions (
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  user_id UUID REFERENCES auth.users(id) ON DELETE CASCADE,
  type VARCHAR(20) NOT NULL CHECK (type IN ('purchase', 'usage', 'bonus',
'refund', 'admin')),
  amount INTEGER NOT NULL,
  balance_before INTEGER NOT NULL,
  balance_after INTEGER NOT NULL,
  feature VARCHAR(100),
  package_id VARCHAR(50) REFERENCES credit_packages(id),
  payment_method VARCHAR(50),
  payment_reference VARCHAR(200),
  status VARCHAR(20) DEFAULT 'completed',
  metadata JSONB DEFAULT '{}',
  created_at TIMESTAMP DEFAULT NOW()
);

-- Index pour performance
```

```
CREATE INDEX idx_transactions_user_date ON credit_transactions(user_id,
created_at DESC);
CREATE INDEX idx_transactions_feature ON credit_transactions(feature);
CREATE INDEX idx_transactions_type ON credit_transactions(type);
CREATE INDEX idx_user_credits_balance ON user_credits(balance);

-- ======================
-- TABLE: payment_transactions
-- ======================
CREATE TABLE payment_transactions (
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  user_id UUID REFERENCES auth.users(id) ON DELETE CASCADE,
  package_id VARCHAR(50) REFERENCES credit_packages(id),
  amount_fcfa INTEGER NOT NULL,
  operator VARCHAR(50) NOT NULL, -- 'airtel', 'moov', 'gtmoney'
  phone_number VARCHAR(20) NOT NULL,
  payment_reference VARCHAR(200) UNIQUE,
  external_reference VARCHAR(200), -- Référence opérateur
  status VARCHAR(20) DEFAULT 'pending', -- 'pending', 'success', 'failed',
'cancelled'
  error_message TEXT,
  metadata JSONB DEFAULT '{}',
  created_at TIMESTAMP DEFAULT NOW(),
  updated_at TIMESTAMP DEFAULT NOW()
);

CREATE INDEX idx_payment_status ON payment_transactions(status);
CREATE INDEX idx_payment_reference ON
payment_transactions(payment_reference);

-- ======================
-- TABLE: loyalty_tiers
-- ======================
CREATE TABLE loyalty_tiers (
  tier VARCHAR(20) PRIMARY KEY,
  min_spent INTEGER NOT NULL,
  bonus_percentage INTEGER DEFAULT 0,
  perks JSONB DEFAULT '[]',
  created_at TIMESTAMP DEFAULT NOW()
);

INSERT INTO loyalty_tiers (tier, min_spent, bonus_percentage, perks) VALUES
('bronze', 0, 0, '[]'::jsonb),
('silver', 5000, 5, '["Support prioritaire"]'::jsonb),
('gold', 15000, 10, '["Support VIP", "Accès beta", "1 business plan gratuit/
mois"]'::jsonb),
('platinum', 50000, 15, '["Support 24/7", "Toutes beta features", "3 business
```

```
plans gratuits/mois", "Consultation 1-on-1"]'::jsonb);

-- ======================
-- FONCTIONS
-- ======================

-- Fonction: Obtenir le solde utilisateur
CREATE OR REPLACE FUNCTION get_user_balance(p_user_id UUID)
RETURNS INTEGER AS $$
DECLARE
  v_balance INTEGER;
BEGIN
  SELECT balance INTO v_balance
  FROM user_credits
  WHERE user_id = p_user_id;

  RETURN COALESCE(v_balance, 0);
END;
$$ LANGUAGE plpgsql;

-- Fonction: Mettre à jour le tier de fidélité
CREATE OR REPLACE FUNCTION update_loyalty_tier(p_user_id UUID)
RETURNS VARCHAR AS $$
DECLARE
  v_lifetime_purchased INTEGER;
  v_new_tier VARCHAR(20);
BEGIN
  SELECT lifetime_purchased INTO v_lifetime_purchased
  FROM user_credits
  WHERE user_id = p_user_id;

  SELECT tier INTO v_new_tier
  FROM loyalty_tiers
  WHERE min_spent <= v_lifetime_purchased
  ORDER BY min_spent DESC
  LIMIT 1;

  UPDATE user_credits
  SET tier = v_new_tier
  WHERE user_id = p_user_id;

  RETURN v_new_tier;
END;
$$ LANGUAGE plpgsql;

-- Fonction: Ajouter des crédits
CREATE OR REPLACE FUNCTION add_credits(
```

```
  p_user_id UUID,
  p_amount INTEGER,
  p_type VARCHAR(20) DEFAULT 'purchase',
  p_package_id VARCHAR(50) DEFAULT NULL,
  p_payment_method VARCHAR(50) DEFAULT NULL,
  p_payment_reference VARCHAR(200) DEFAULT NULL,
  p_metadata JSONB DEFAULT '{}'
)
RETURNS JSONB AS $$
DECLARE
  v_balance_before INTEGER;
  v_balance_after INTEGER;
  v_transaction_id UUID;
BEGIN
  -- Récupérer ou créer l'entrée user_credits
  INSERT INTO user_credits (user_id, balance)
  VALUES (p_user_id, 0)
  ON CONFLICT (user_id) DO NOTHING;

  -- Récupérer le solde actuel
  SELECT balance INTO v_balance_before
  FROM user_credits
  WHERE user_id = p_user_id;

  v_balance_after := v_balance_before + p_amount;

  -- Mettre à jour le solde
  UPDATE user_credits
  SET
    balance = v_balance_after,
    lifetime_purchased = lifetime_purchased + p_amount,
    last_purchase_at = CASE WHEN p_type = 'purchase' THEN NOW() ELSE
last_purchase_at END,
    updated_at = NOW()
  WHERE user_id = p_user_id;

  -- Enregistrer la transaction
  INSERT INTO credit_transactions (
    user_id, type, amount, balance_before, balance_after,
    package_id, payment_method, payment_reference, metadata
  )
  VALUES (
    p_user_id, p_type, p_amount, v_balance_before, v_balance_after,
    p_package_id, p_payment_method, p_payment_reference, p_metadata
  )
  RETURNING id INTO v_transaction_id;
```

```
  -- Mettre à jour le tier
  PERFORM update_loyalty_tier(p_user_id);

  RETURN jsonb_build_object(
    'success', true,
    'transaction_id', v_transaction_id,
    'balance_before', v_balance_before,
    'balance_after', v_balance_after,
    'amount_added', p_amount
  );
END;
$$ LANGUAGE plpgsql;

-- Fonction: Consommer des crédits
CREATE OR REPLACE FUNCTION consume_credits(
  p_user_id UUID,
  p_amount INTEGER,
  p_feature VARCHAR(100),
  p_metadata JSONB DEFAULT '{}'
)
RETURNS JSONB AS $$
DECLARE
  v_balance_before INTEGER;
  v_balance_after INTEGER;
  v_transaction_id UUID;
BEGIN
  -- Récupérer le solde actuel
  SELECT balance INTO v_balance_before
  FROM user_credits
  WHERE user_id = p_user_id;

  -- Vérifier si l'utilisateur a assez de crédits
  IF v_balance_before IS NULL OR v_balance_before < p_amount THEN
    RETURN jsonb_build_object(
      'success', false,
      'error', 'insufficient_credits',
      'required', p_amount,
      'available', COALESCE(v_balance_before, 0),
      'missing', p_amount - COALESCE(v_balance_before, 0)
    );
  END IF;

  v_balance_after := v_balance_before - p_amount;

  -- Mettre à jour le solde
  UPDATE user_credits
  SET
```

```
    balance = v_balance_after,
    lifetime_spent = lifetime_spent + p_amount,
    last_usage_at = NOW(),
    updated_at = NOW()
  WHERE user_id = p_user_id;

  -- Enregistrer la transaction
  INSERT INTO credit_transactions (
    user_id, type, amount, balance_before, balance_after,
    feature, metadata
  )
  VALUES (
    p_user_id, 'usage', -p_amount, v_balance_before, v_balance_after,
    p_feature, p_metadata
  )
  RETURNING id INTO v_transaction_id;

  RETURN jsonb_build_object(
    'success', true,
    'transaction_id', v_transaction_id,
    'balance_before', v_balance_before,
    'balance_after', v_balance_after,
    'amount_consumed', p_amount,
    'feature', p_feature
  );
END;
$$ LANGUAGE plpgsql;

-- ======================
-- RLS (Row Level Security)
-- ======================

ALTER TABLE user_credits ENABLE ROW LEVEL SECURITY;
ALTER TABLE credit_transactions ENABLE ROW LEVEL SECURITY;
ALTER TABLE payment_transactions ENABLE ROW LEVEL SECURITY;

-- Policies user_credits
CREATE POLICY "Users can view own credits"
  ON user_credits FOR SELECT
  USING (auth.uid() = user_id);

-- Policies credit_transactions
CREATE POLICY "Users can view own transactions"
  ON credit_transactions FOR SELECT
  USING (auth.uid() = user_id);

-- Policies payment_transactions
```

```
CREATE POLICY "Users can view own payments"
  ON payment_transactions FOR SELECT
  USING (auth.uid() = user_id);

-- Policies credit_packages (lecture publique)
ALTER TABLE credit_packages ENABLE ROW LEVEL SECURITY;
CREATE POLICY "Packages are viewable by everyone"
  ON credit_packages FOR SELECT
  USING (active = true);

-- ======================
-- TRIGGERS
-- ======================

-- Trigger: Mettre à jour updated_at
CREATE OR REPLACE FUNCTION update_updated_at()
RETURNS TRIGGER AS $$
BEGIN
  NEW.updated_at = NOW();
  RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER update_user_credits_updated_at
  BEFORE UPDATE ON user_credits
  FOR EACH ROW
  EXECUTE FUNCTION update_updated_at();

CREATE TRIGGER update_payment_transactions_updated_at
  BEFORE UPDATE ON payment_transactions
  FOR EACH ROW
  EXECUTE FUNCTION update_updated_at();

-- ======================
-- VUES UTILES
-- ======================

-- Vue: Statistiques utilisateur
CREATE OR REPLACE VIEW user_credit_stats AS
SELECT
  uc.user_id,
  uc.balance,
  uc.lifetime_purchased,
  uc.lifetime_spent,
  uc.tier,
  lt.bonus_percentage,
  lt.perks,
```

```
  uc.last_purchase_at,
  uc.last_usage_at,
  COUNT(DISTINCT ct.id) FILTER (WHERE ct.type = 'purchase') as
total_purchases,
  COUNT(DISTINCT ct.id) FILTER (WHERE ct.type = 'usage') as total_usages,
  COALESCE(SUM(ct.amount) FILTER (WHERE ct.type = 'purchase'), 0) as
total_credits_purchased,
  COALESCE(ABS(SUM(ct.amount)) FILTER (WHERE ct.type = 'usage'), 0) as
total_credits_spent
FROM user_credits uc
LEFT JOIN loyalty_tiers lt ON uc.tier = lt.tier
LEFT JOIN credit_transactions ct ON uc.user_id = ct.user_id
GROUP BY uc.user_id, uc.balance, uc.lifetime_purchased, uc.lifetime_spent,
         uc.tier, lt.bonus_percentage, lt.perks, uc.last_purchase_at,
uc.last_usage_at;

COMMENT ON VIEW user_credit_stats IS 'Vue consolidée des statistiques de
crédits par utilisateur';
```

2️⃣ CONSTANTES ET CONFIGURATION

```javascript
// src/lib/constants.js

export const FEATURE_COSTS = {
  // Fonctions principales
  business_plan_full: 850,
  chat_message: 200,
  actu_plus: 320,
  opportunity_analysis: 180,
  training_module: 150,
  action_plan: 150,
  skills_test: 100,
  project_framework: 75,
  custom_summary: 45,
  letter_generation: 45,
  audio_survey: 20,

  // Futures fonctionnalités
  business_intelligence_report: 500,
  market_research: 400,
  competitive_analysis: 300,
};
```

```
export const FEATURE_NAMES = {
  business_plan_full: 'Business Plan Complet',
  chat_message: 'Message Chat IA',
  actu_plus: 'Analyse Actu Plus',
  opportunity_analysis: 'Analyse d\'Opportunités',
  training_module: 'Module de Formation',
  action_plan: 'Plan d\'Action',
  skills_test: 'Test de Compétences',
  project_framework: 'Framework Projet',
  custom_summary: 'Résumé Personnalisé',
  letter_generation: 'Génération de Courrier',
  audio_survey: 'Sondage Audio',
  business_intelligence_report: 'Rapport Business Intelligence',
  market_research: 'Étude de Marché',
  competitive_analysis: 'Analyse Concurrentielle',
};

export const PAYMENT_OPERATORS = {
  airtel: {
    name: 'Airtel Money',
    code: 'airtel',
    logo: '/assets/operators/airtel.png',
    ussdCode: '*155#',
  },
  moov: {
    name: 'Moov Money',
    code: 'moov',
    logo: '/assets/operators/moov.png',
    ussdCode: '*155#',
  },
  gtmoney: {
    name: 'Gabon Telecom Money',
    code: 'gtmoney',
    logo: '/assets/operators/gtmoney.png',
    ussdCode: '*123#',
  },
};

export const LOYALTY_TIERS = {
  bronze: {
    name: 'Bronze',
    minSpent: 0,
    bonus: 0,
    color: '#CD7F32',
    icon: ' 🥉 ',
  },
```

```
  silver: {
    name: 'Silver',
    minSpent: 5000,
    bonus: 5,
    color: '#C0C0C0',
    icon: ' 🥈 ',
  },
  gold: {
    name: 'Gold',
    minSpent: 15000,
    bonus: 10,
    color: '#FFD700',
    icon: ' 🥇 ',
  },
  platinum: {
    name: 'Platinum',
    minSpent: 50000,
    bonus: 15,
    color: '#E5E4E2',
    icon: '💎',
  },
};

export const CREDIT_THRESHOLDS = {
  critical: 200,   // Alerte rouge
  low: 500,        // Alerte orange
  comfortable: 1000, // Vert
};
```

3️⃣ BIBLIOTHÈQUE CRÉDITS

```javascript
// src/lib/credits.js

import { supabase } from './supabase';
import { FEATURE_COSTS, CREDIT_THRESHOLDS } from './constants';

/**
 * Obtenir le solde de crédits de l'utilisateur
 */
export async function getUserBalance(userId) {
  const { data, error } = await supabase
    .from('user_credits')
    .select('*')
```

```
    .eq('user_id', userId)
    .single();

  if (error && error.code !== 'PGRST116') {
    throw error;
  }

  return data || { balance: 0, tier: 'bronze' };
}

/**
 * Obtenir les statistiques complètes de l'utilisateur
 */
export async function getUserStats(userId) {
  const { data, error } = await supabase
    .from('user_credit_stats')
    .select('*')
    .eq('user_id', userId)
    .single();

  if (error) throw error;
  return data;
}

/**
 * Vérifier si l'utilisateur a assez de crédits
 */
export async function hasEnoughCredits(userId, feature) {
  const cost = FEATURE_COSTS[feature];
  if (!cost) {
    throw new Error(`Feature inconnu: ${feature}`);
  }

  const { balance } = await getUserBalance(userId);
  return balance >= cost;
}

/**
 * Consommer des crédits pour une fonctionnalité
 */
export async function consumeCredits(userId, feature, metadata = {}) {
  const cost = FEATURE_COSTS[feature];
  if (!cost) {
    throw new Error(`Feature inconnu: ${feature}`);
  }

  const { data, error } = await supabase.rpc('consume_credits', {
```

```
    p_user_id: userId,
    p_amount: cost,
    p_feature: feature,
    p_metadata: metadata,
  });

  if (error) throw error;

  if (!data.success) {
    const err = new Error(data.error || 'Erreur lors de la consommation des
crédits');
    err.code = data.error;
    err.details = data;
    throw err;
  }

  return data;
}

/**
 * Ajouter des crédits (achat, bonus, etc.)
 */
export async function addCredits(userId, amount, options = {}) {
  const {
    type = 'purchase',
    packageId = null,
    paymentMethod = null,
    paymentReference = null,
    metadata = {},
  } = options;

  const { data, error } = await supabase.rpc('add_credits', {
    p_user_id: userId,
    p_amount: amount,
    p_type: type,
    p_package_id: packageId,
    p_payment_method: paymentMethod,
    p_payment_reference: paymentReference,
    p_metadata: metadata,
  });

  if (error) throw error;
  return data;
}

/**
 * Obtenir l'historique des transactions
```

```
 */
export async function getTransactionHistory(userId, options = {}) {
  const {
    limit = 50,
    offset = 0,
    type = null,
    startDate = null,
    endDate = null,
  } = options;

  let query = supabase
    .from('credit_transactions')
    .select('*')
    .eq('user_id', userId)
    .order('created_at', { ascending: false })
    .range(offset, offset + limit - 1);

  if (type) {
    query = query.eq('type', type);
  }

  if (startDate) {
    query = query.gte('created_at', startDate);
  }

  if (endDate) {
    query = query.lte('created_at', endDate);
  }

  const { data, error } = await query;
  if (error) throw error;

  return data;
}

/**
 * Obtenir tous les packages disponibles
 */
export async function getCreditPackages() {
  const { data, error } = await supabase
    .from('credit_packages')
    .select('*')
    .eq('active', true)
    .order('sort_order');

  if (error) throw error;
  return data;
```

```
}

/**
 * Obtenir un package spécifique
 */
export async function getPackage(packageId) {
  const { data, error } = await supabase
    .from('credit_packages')
    .select('*')
    .eq('id', packageId)
    .single();

  if (error) throw error;
  return data;
}

/**
 * Obtenir le niveau d'urgence du solde
 */
export function getBalanceUrgency(balance) {
  if (balance < CREDIT_THRESHOLDS.critical) {
    return {
      level: 'critical',
      color: '#EF4444',
      icon: '⚠️',
      message: 'Crédits presque épuisés !',
      cta: 'Recharger maintenant',
    };
  }

  if (balance < CREDIT_THRESHOLDS.low) {
    return {
      level: 'low',
      color: '#F59E0B',
      icon: '⚡',
      message: 'Pensez à recharger bientôt',
      cta: 'Voir les offres',
    };
  }

  if (balance < CREDIT_THRESHOLDS.comfortable) {
    return {
      level: 'moderate',
      color: '#10B981',
      icon: '✅',
      message: 'Solde correct',
      cta: null,
```

```
    };
  }

  return {
    level: 'good',
    color: '#10B981',
    icon: '💎',
    message: 'Solde excellent',
    cta: null,
  };
}

/**
 * Calculer combien d'utilisations sont possibles avec un solde donné
 */
export function calculateUsageCapacity(balance, feature) {
  const cost = FEATURE_COSTS[feature];
  if (!cost) return 0;
  return Math.floor(balance / cost);
}

/**
 * Calculer le coût total pour plusieurs actions
 */
export function calculateTotalCost(features) {
  return features.reduce((total, feature) => {
    const cost = FEATURE_COSTS[feature.name] || 0;
    const quantity = feature.quantity || 1;
    return total + (cost * quantity);
  }, 0);
}
```

4️⃣ INTÉGRATION MOBILE MONEY

```javascript
// src/lib/momo-payment.js

import { supabase } from './supabase';

/**
 * Configuration des opérateurs Mobile Money au Gabon
 * À adapter selon les APIs réelles de chaque opérateur
 */
```

```
const MOMO_CONFIGS = {
  airtel: {
    apiUrl: process.env.NEXT_PUBLIC_AIRTEL_API_URL,
    apiKey: process.env.AIRTEL_API_KEY,
    merchantId: process.env.AIRTEL_MERCHANT_ID,
  },
  moov: {
    apiUrl: process.env.NEXT_PUBLIC_MOOV_API_URL,
    apiKey: process.env.MOOV_API_KEY,
    merchantId: process.env.MOOV_MERCHANT_ID,
  },
  gtmoney: {
    apiUrl: process.env.NEXT_PUBLIC_GTMONEY_API_URL,
    apiKey: process.env.GTMONEY_API_KEY,
    merchantId: process.env.GTMONEY_MERCHANT_ID,
  },
};

/**
 * Générer une référence de paiement unique
 */
function generatePaymentReference(userId, packageId) {
  const timestamp = Date.now();
  const random = Math.random().toString(36).substring(7);
  return `G247_${userId.substring(0, 8)}_${packageId}_${timestamp}_$
{random}`.toUpperCase();
}

/**
 * Initier un paiement Mobile Money
 */
export async function initiateMobileMoneyPayment({
  userId,
  packageId,
  amount,
  phoneNumber,
  operator,
}) {
  try {
    // Validation
    if (!MOMO_CONFIGS[operator]) {
      throw new Error(`Opérateur non supporté: ${operator}`);
    }

    // Générer référence
    const reference = generatePaymentReference(userId, packageId);
```

```
// Créer la transaction en attente
const { data: transaction, error: transactionError } = await supabase
  .from('payment_transactions')
  .insert({
    user_id: userId,
    package_id: packageId,
    amount_fcfa: amount,
    operator: operator,
    phone_number: phoneNumber,
    payment_reference: reference,
    status: 'pending',
    metadata: {
      initiated_at: new Date().toISOString(),
    },
  })
  .select()
  .single();

if (transactionError) throw transactionError;

// Appeler l'API de l'opérateur
const paymentResult = await callOperatorAPI(operator, {
  amount,
  phoneNumber,
  reference,
  description: `Achat crédits Gabon 24/7 - Package ${packageId}`,
});

// Mettre à jour avec la référence externe
await supabase
  .from('payment_transactions')
  .update({
    external_reference: paymentResult.externalReference,
    metadata: {
      ...transaction.metadata,
      operator_response: paymentResult,
    },
  })
  .eq('id', transaction.id);

return {
  success: true,
  transactionId: transaction.id,
  reference: reference,
  externalReference: paymentResult.externalReference,
  status: 'pending',
```

```
      message: `Composez ${MOMO_CONFIGS[operator].ussdCode || '*155#'}
pour confirmer le paiement`,
      instructions: getPaymentInstructions(operator, phoneNumber),
    };
  } catch (error) {
    console.error('Erreur initiation paiement:', error);
    throw error;
  }
}

/**
 * Appeler l'API de l'opérateur Mobile Money
 * NOTE: Ceci est un exemple générique à adapter selon chaque opérateur
 */
async function callOperatorAPI(operator, paymentData) {
  const config = MOMO_CONFIGS[operator];

  // EXEMPLE GÉNÉRIQUE - À ADAPTER
  const response = await fetch(`${config.apiUrl}/v1/payments/initiate`, {
    method: 'POST',
    headers: {
      'Content-Type': 'application/json',
      'Authorization': `Bearer ${config.apiKey}`,
      'X-Merchant-Id': config.merchantId,
    },
    body: JSON.stringify({
      amount: paymentData.amount,
      phone: paymentData.phoneNumber,
      reference: paymentData.reference,
      description: paymentData.description,
      callback_url: `${process.env.NEXT_PUBLIC_APP_URL}/api/credits/
webhook`,
    }),
  });

  if (!response.ok) {
    const error = await response.json();
    throw new Error(error.message || 'Erreur API opérateur');
  }

  const result = await response.json();
  return {
    externalReference: result.transaction_id || result.reference,
    status: result.status,
    ...result,
  };
}
```

```
/**
 * Vérifier le statut d'un paiement
 */
export async function checkPaymentStatus(transactionId) {
  const { data: transaction, error } = await supabase
    .from('payment_transactions')
    .select('*')
    .eq('id', transactionId)
    .single();

  if (error) throw error;

  // Si déjà traité, retourner le statut
  if (transaction.status !== 'pending') {
    return {
      status: transaction.status,
      transaction,
    };
  }

  // Sinon, vérifier auprès de l'opérateur
  try {
    const config = MOMO_CONFIGS[transaction.operator];
    const response = await fetch(
      `${config.apiUrl}/v1/payments/${transaction.external_reference}/status`,
      {
        headers: {
          'Authorization': `Bearer ${config.apiKey}`,
          'X-Merchant-Id': config.merchantId,
        },
      }
    );

    const result = await response.json();

    // Mettre à jour le statut si changé
    if (result.status !== transaction.status) {
      await updatePaymentStatus(transactionId, result.status, result);
    }

    return {
      status: result.status,
      transaction: {
        ...transaction,
        status: result.status,
      },
```

```
    };
  } catch (error) {
    console.error('Erreur vérification statut:', error);
    return {
      status: transaction.status,
      transaction,
    };
  }
}

/**
 * Mettre à jour le statut d'un paiement
 */
export async function updatePaymentStatus(transactionId, status,
operatorData = {}) {
  const { data: transaction, error: fetchError } = await supabase
    .from('payment_transactions')
    .select('*')
    .eq('id', transactionId)
    .single();

  if (fetchError) throw fetchError;

  // Mettre à jour le statut
  const { error: updateError } = await supabase
    .from('payment_transactions')
    .update({
      status,
      metadata: {
        ...transaction.metadata,
        status_updated_at: new Date().toISOString(),
        operator_data: operatorData,
      },
    })
    .eq('id', transactionId);

  if (updateError) throw updateError;

  // Si succès, créditer l'utilisateur
  if (status === 'success') {
    await creditUserAfterPayment(transaction);
  }

  return { success: true, status };
}

/**
```

```
 * Créditer l'utilisateur après paiement réussi
 */
async function creditUserAfterPayment(transaction) {
  const { data: packageData, error: packageError } = await supabase
    .from('credit_packages')
    .select('*')
    .eq('id', transaction.package_id)
    .single();

  if (packageError) throw packageError;

  // Calculer le bonus selon le tier
  const { data: userCredits } = await supabase
    .from('user_credits')
    .select('tier')
    .eq('user_id', transaction.user_id)
    .single();

  let bonusPercentage = 0;
  if (userCredits) {
    const { data: tierData } = await supabase
      .from('loyalty_tiers')
      .select('bonus_percentage')
      .eq('tier', userCredits.tier)
      .single();
    bonusPercentage = tierData?.bonus_percentage || 0;
  }

  const loyaltyBonus = Math.floor(packageData.total_credits *
(bonusPercentage / 100));
  const totalCredits = packageData.total_credits + loyaltyBonus;

  // Ajouter les crédits
  const { data, error } = await supabase.rpc('add_credits', {
    p_user_id: transaction.user_id,
    p_amount: totalCredits,
    p_type: 'purchase',
    p_package_id: transaction.package_id,
    p_payment_method: transaction.operator,
    p_payment_reference: transaction.payment_reference,
    p_metadata: {
      base_credits: packageData.base_credits,
      package_bonus: packageData.bonus_credits,
      loyalty_bonus: loyaltyBonus,
      total_credits: totalCredits,
    },
  });
```

```
  if (error) throw error;

  // TODO: Envoyer notification push/email
  await sendPurchaseNotification(transaction.user_id, {
    credits: totalCredits,
    packageName: packageData.name,
  });

  return data;
}

/**
 * Instructions de paiement selon l'opérateur
 */
function getPaymentInstructions(operator, phoneNumber) {
  const instructions = {
    airtel: [
      `1. Composez *155# sur votre téléphone ${phoneNumber}`,
      '2. Sélectionnez "Payer Marchand"',
      '3. Entrez le code marchand affiché',
      '4. Entrez le montant',
      '5. Validez avec votre code PIN',
      '6. Vous recevrez une confirmation SMS',
    ],
    moov: [
      `1. Composez *155# sur votre téléphone ${phoneNumber}`,
      '2. Sélectionnez "Paiements"',
      '3. Choisissez "Payer un marchand"',
      '4. Entrez le code marchand',
      '5. Entrez le montant',
      '6. Confirmez avec votre code secret',
    ],
    gtmoney: [
      `1. Composez *123# sur votre téléphone ${phoneNumber}`,
      '2. Sélectionnez "Paiements"',
      '3. Entrez le numéro marchand',
      '4. Entrez le montant',
      '5. Validez avec votre code PIN',
    ],
  };

  return instructions[operator] || [];
}

/**
 * Envoyer notification d'achat
```

```
 */
async function sendPurchaseNotification(userId, data) {
  // TODO: Implémenter l'envoi de notification
  // Via push notification, email, SMS, etc.
  console.log('Notification achat:', userId, data);
}

/**
 * Gérer le webhook de callback de l'opérateur
 */
export async function handlePaymentWebhook(payload) {
  try {
    // Valider la signature du webhook si applicable
    // validateWebhookSignature(payload);

    const { reference, status, external_reference } = payload;

    // Trouver la transaction
    const { data: transaction, error } = await supabase
      .from('payment_transactions')
      .select('*')
      .eq('payment_reference', reference)
      .single();

    if (error) throw error;

    // Mettre à jour le statut
    await updatePaymentStatus(transaction.id, status, payload);

    return { success: true };
  } catch (error) {
    console.error('Erreur webhook:', error);
    throw error;
  }
}
```

5️⃣ HOOK REACT - useCredits

```javascript
// src/hooks/useCredits.js

import { useState, useEffect, useCallback } from 'react';
import { useAuth } from './useAuth'; // Votre hook d'authentification
import {
```

```
  getUserBalance,
  getUserStats,
  consumeCredits,
  getTransactionHistory,
  getCreditPackages,
  getBalanceUrgency,
  calculateUsageCapacity,
} from '../lib/credits';
import { FEATURE_COSTS, FEATURE_NAMES } from '../lib/constants';

export function useCredits() {
  const { user } = useAuth();
  const [credits, setCredits] = useState(null);
  const [stats, setStats] = useState(null);
  const [packages, setPackages] = useState([]);
  const [history, setHistory] = useState([]);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);

  // Charger les données initiales
  useEffect(() => {
    if (user) {
      loadCreditsData();
      loadPackages();
    }
  }, [user]);

  const loadCreditsData = useCallback(async () => {
    try {
      setLoading(true);
      const [balanceData, statsData] = await Promise.all([
        getUserBalance(user.id),
        getUserStats(user.id),
      ]);
      setCredits(balanceData);
      setStats(statsData);
      setError(null);
    } catch (err) {
      console.error('Erreur chargement crédits:', err);
      setError(err.message);
    } finally {
      setLoading(false);
    }
  }, [user]);

  const loadPackages = useCallback(async () => {
    try {
```

```
    const data = await getCreditPackages();
    setPackages(data);
  } catch (err) {
    console.error('Erreur chargement packages:', err);
  }
}, []);

const loadHistory = useCallback(async (options = {}) => {
  try {
    const data = await getTransactionHistory(user.id, options);
    setHistory(data);
  } catch (err) {
    console.error('Erreur chargement historique:', err);
  }
}, [user]);

// Consommer des crédits
const consume = useCallback(async (feature, metadata = {}) => {
  try {
    const result = await consumeCredits(user.id, feature, metadata);

    // Recharger le solde
    await loadCreditsData();

    return result;
  } catch (err) {
    if (err.code === 'insufficient_credits') {
      // Gérer le cas de crédits insuffisants
      return {
        success: false,
        error: 'insufficient_credits',
        details: err.details,
      };
    }
    throw err;
  }
}, [user, loadCreditsData]);

// Vérifier si une action est possible
const canAfford = useCallback((feature) => {
  if (!credits) return false;
  const cost = FEATURE_COSTS[feature];
  return credits.balance >= cost;
}, [credits]);

// Obtenir le coût d'une fonctionnalité
const getCost = useCallback((feature) => {
```

```
    return FEATURE_COSTS[feature] || 0;
  }, []);

  // Obtenir le nom d'une fonctionnalité
  const getFeatureName = useCallback((feature) => {
    return FEATURE_NAMES[feature] || feature;
  }, []);

  // Obtenir l'urgence du solde
  const urgency = credits ? getBalanceUrgency(credits.balance) : null;

  // Calculer la capacité d'usage
  const getCapacity = useCallback((feature) => {
    if (!credits) return 0;
    return calculateUsageCapacity(credits.balance, feature);
  }, [credits]);

  return {
    // État
    credits,
    stats,
    packages,
    history,
    loading,
    error,
    urgency,

    // Méthodes
    consume,
    canAfford,
    getCost,
    getFeatureName,
    getCapacity,
    refresh: loadCreditsData,
    loadHistory,
  };
}
```

Je continue avec les composants React dans le prochain message. Voulez-vous que je poursuive ?

🚀 CODE COMPLET - PARTIE 2 : COMPOSANTS REACT

6️⃣ COMPOSANT - Widget Crédits

```jsx
// src/components/CreditWidget.jsx

import { useState } from 'react';
import { useCredits } from '../hooks/useCredits';
import { useRouter } from 'next/router';
import { LOYALTY_TIERS } from '../lib/constants';

export default function CreditWidget({ compact = false }) {
  const { credits, urgency, loading } = useCredits();
  const router = useRouter();
  const [showDropdown, setShowDropdown] = useState(false);

  if (loading || !credits) {
    return (
      <div className="credit-widget-skeleton">
        <div className="skeleton-pulse"></div>
      </div>
    );
  }

  const tier = LOYALTY_TIERS[credits.tier] || LOYALTY_TIERS.bronze;

  return (
    <div className="credit-widget-container">
      <div
        className={`credit-widget ${urgency?.level}`}
        onClick={() => setShowDropdown(!showDropdown)}
        style={{ borderColor: urgency?.color }}
      >
        {/* Balance principale */}
        <div className="credit-balance">
          <span className="credit-icon">{tier.icon}</span>
          <div className="credit-amount">
            <span className="credit-number">
              {credits.balance.toLocaleString()}
            </span>
            <span className="credit-label">crédits</span>
          </div>
        </div>

        {/* Indicateur d'urgence */}
        {urgency && urgency.level !== 'good' && (
          <div className="urgency-indicator" style={{ color: urgency.color }}>
            <span className="urgency-icon">{urgency.icon}</span>
            {!compact && (
```

```
        <span className="urgency-message">{urgency.message}</span>
      )}
    </div>
  )}

  {/* Bouton d'action */}
  {urgency?.cta && (
    <button
      className="credit-cta"
      onClick={(e) => {
        e.stopPropagation();
        router.push('/credits/buy');
      }}
    >
      {urgency.cta}
    </button>
  )}
</div>

{/* Dropdown détails */}
{showDropdown && !compact && (
  <div className="credit-dropdown">
    <div className="dropdown-header">
      <h4>Mes Crédits</h4>
      <button
        className="close-btn"
        onClick={() => setShowDropdown(false)}
      >
        ✕
      </button>
    </div>

    <div className="dropdown-content">
      {/* Tier actuel */}
      <div className="tier-badge" style={{ borderColor: tier.color }}>
        <span className="tier-icon">{tier.icon}</span>
        <div className="tier-info">
          <span className="tier-name">{tier.name}</span>
          <span className="tier-bonus">+{tier.bonus}% bonus</span>
        </div>
      </div>

      {/* Statistiques */}
      <div className="credit-stats">
        <div className="stat-item">
          <span className="stat-label">Total acheté</span>
          <span className="stat-value">
```

```
              {credits.lifetime_purchased?.toLocaleString() || 0}
            </span>
          </div>
          <div className="stat-item">
            <span className="stat-label">Total utilisé</span>
            <span className="stat-value">
              {credits.lifetime_spent?.toLocaleString() || 0}
            </span>
          </div>
        </div>

        {/* Actions */}
        <div className="dropdown-actions">
          <button
            className="btn-primary"
            onClick={() => router.push('/credits/buy')}
          >
            💎 Acheter des crédits
          </button>
          <button
            className="btn-secondary"
            onClick={() => router.push('/credits/history')}
          >
            📊 Historique
          </button>
        </div>
      </div>
    </div>
  )}
 </div>
);
}
```

7️⃣ COMPOSANT - Page d'Achat de Crédits

```jsx
// src/pages/credits/buy.jsx

import { useState, useEffect } from 'react';
import { useCredits } from '../../hooks/useCredits';
import { useAuth } from '../../hooks/useAuth';
import { initiateMobileMoneyPayment } from '../../lib/momo-payment';
import { PAYMENT_OPERATORS } from '../../lib/constants';
import CreditWidget from '../../components/CreditWidget';
```

```
export default function BuyCreditsPage() {
  const { user } = useAuth();
  const { packages, credits, loading } = useCredits();
  const [selectedPackage, setSelectedPackage] = useState(null);
  const [phoneNumber, setPhoneNumber] = useState('');
  const [operator, setOperator] = useState('airtel');
  const [processing, setProcessing] = useState(false);
  const [error, setError] = useState(null);
  const [paymentInstructions, setPaymentInstructions] = useState(null);

  useEffect(() => {
    // Pré-sélectionner le package le plus populaire
    if (packages.length > 0 && !selectedPackage) {
      const popular = packages.find(p => p.popular) || packages[0];
      setSelectedPackage(popular);
    }
  }, [packages]);

  const handlePurchase = async () => {
    try {
      setError(null);
      setProcessing(true);

      // Validation
      if (!phoneNumber || phoneNumber.length < 8) {
        throw new Error('Numéro de téléphone invalide');
      }

      if (!selectedPackage) {
        throw new Error('Veuillez sélectionner un package');
      }

      // Initier le paiement
      const result = await initiateMobileMoneyPayment({
        userId: user.id,
        packageId: selectedPackage.id,
        amount: selectedPackage.price_fcfa,
        phoneNumber: phoneNumber,
        operator: operator,
      });

      if (result.success) {
        setPaymentInstructions({
          message: result.message,
          instructions: result.instructions,
          reference: result.reference,
```

```
      });

      // Démarrer le polling du statut
      startPaymentPolling(result.transactionId);
    }
  } catch (err) {
    console.error('Erreur achat:', err);
    setError(err.message);
  } finally {
    setProcessing(false);
  }
};

const startPaymentPolling = (transactionId) => {
  // TODO: Implémenter polling du statut de paiement
  // Vérifier toutes les 5 secondes pendant 5 minutes
};

if (loading) {
  return <div className="loading">Chargement...</div>;
}

return (
  <div className="buy-credits-page">
    {/* Header */}
    <div className="page-header">
      <h1>💎 Acheter des Crédits</h1>
      <CreditWidget compact />
    </div>

    {/* Grille des packages */}
    <div className="packages-grid">
      {packages.map((pkg) => (
        <div
          key={pkg.id}
          className={`package-card ${selectedPackage?.id === pkg.id ?
'selected' : ''} ${pkg.popular ? 'popular' : ''}`}
          onClick={() => setSelectedPackage(pkg)}
        >
          {pkg.popular && (
            <div className="popular-badge">⭐ PLUS POPULAIRE</div>
          )}

          <div className="package-header">
            <h3>{pkg.name}</h3>
            <div className="package-price">
              <span className="price-amount">{pkg.price_fcfa.toLocaleString()}
```

```
</span>
            <span className="price-currency">FCFA</span>
          </div>
        </div>

        <div className="package-credits">
          <div className="credits-main">
            <span className="credits-icon">💎</span>
            <span className="credits-amount">
              {pkg.base_credits.toLocaleString()}
            </span>
            <span className="credits-label">crédits</span>
          </div>

          {pkg.bonus_credits > 0 && (
            <div className="credits-bonus">
              + {pkg.bonus_credits.toLocaleString()} crédits bonus 🎁
            </div>
          )}

          <div className="credits-total">
            Total: <strong>{pkg.total_credits.toLocaleString()}</strong> crédits
          </div>
        </div>

        <div className="package-features">
          {pkg.features.map((feature, index) => (
            <div key={index} className="feature-item">
              {feature}
            </div>
          ))}
        </div>

        <div className="package-value">
          <span className="value-label">Prix par crédit:</span>
          <span className="value-amount">
            {(pkg.price_fcfa / pkg.total_credits).toFixed(2)} FCFA
          </span>
        </div>
      </div>
    ))}
  </div>

  {/* Formulaire de paiement */}
  {selectedPackage && !paymentInstructions && (
    <div className="payment-form">
      <h2>💳 Informations de Paiement</h2>
```

```
        <div className="selected-package-summary">
          <div className="summary-item">
            <span>Package:</span>
            <strong>{selectedPackage.name}</strong>
          </div>
          <div className="summary-item">
            <span>Crédits:</span>
            <strong>{selectedPackage.total_credits.toLocaleString()}</strong>
          </div>
          <div className="summary-item total">
            <span>Total à payer:</span>
            <strong>{selectedPackage.price_fcfa.toLocaleString()} FCFA</strong>
          </div>
        </div>

        {/* Sélection opérateur */}
        <div className="form-group">
          <label>Opérateur Mobile Money</label>
          <div className="operator-selection">
            {Object.values(PAYMENT_OPERATORS).map((op) => (
              <div
                key={op.code}
                className={`operator-card ${operator === op.code ? 'selected' : ''}
`}
                onClick={() => setOperator(op.code)}
              >
                <img src={op.logo} alt={op.name} />
                <span>{op.name}</span>
              </div>
            ))}
          </div>
        </div>

        {/* Numéro de téléphone */}
        <div className="form-group">
          <label>Numéro de téléphone</label>
          <input
            type="tel"
            placeholder="Ex: 06 12 34 56 78"
            value={phoneNumber}
            onChange={(e) => setPhoneNumber(e.target.value)}
            className="phone-input"
          />
          <small className="input-help">
            Le numéro {PAYMENT_OPERATORS[operator].name} à débiter
          </small>
```

```
        </div>

        {/* Erreur */}
        {error && (
          <div className="error-message">
            ⚠️ {error}
          </div>
        )}

        {/* Bouton d'achat */}
        <button
          className="btn-purchase"
          onClick={handlePurchase}
          disabled={processing || !phoneNumber}
        >
          {processing ? (
            <>
              <span className="spinner"></span>
              Traitement en cours...
            </>
          ) : (
            <>
              🛒 Acheter {selectedPackage.total_credits.toLocaleString()} crédits
            </>
          )}
        </button>

        <div className="payment-security">
          🔒 Paiement 100% sécurisé via Mobile Money
        </div>
      </div>
    )}

    {/* Instructions de paiement */}
    {paymentInstructions && (
      <div className="payment-instructions">
        <div className="instructions-header">
          <h2>✅ Paiement Initié</h2>
          <p>{paymentInstructions.message}</p>
        </div>

        <div className="reference-box">
          <span className="reference-label">Référence:</span>
          <code className="reference-code">{paymentInstructions.reference}</
code>
        </div>
```

```
    <div className="instructions-steps">
      <h3>📱 Étapes à suivre:</h3>
      <ol>
        {paymentInstructions.instructions.map((step, index) => (
          <li key={index}>{step}</li>
        ))}
      </ol>
    </div>

    <div className="waiting-confirmation">
      <div className="spinner-large"></div>
      <p>En attente de la confirmation de paiement...</p>
    </div>

    <button
      className="btn-secondary"
      onClick={() => setPaymentInstructions(null)}
    >
      Annuler
    </button>
  </div>
)}

{/* Avantages */}
<div className="benefits-section">
  <h2>🎁 Pourquoi Acheter des Crédits ?</h2>
  <div className="benefits-grid">
    <div className="benefit-card">
      <div className="benefit-icon">⚡</div>
      <h3>Accès Instantané</h3>
      <p>Utilisez vos crédits immédiatement après l'achat</p>
    </div>
    <div className="benefit-card">
      <div className="benefit-icon">🎯</div>
      <h3>Aucun Engagement</h3>
      <p>Payez uniquement pour ce que vous utilisez</p>
    </div>
    <div className="benefit-card">
      <div className="benefit-icon">💰</div>
      <h3>Bonus Progressifs</h3>
      <p>Plus vous achetez, plus vous gagnez de bonus</p>
    </div>
    <div className="benefit-card">
      <div className="benefit-icon">🏆</div>
      <h3>Programme Fidélité</h3>
      <p>Débloquez des avantages exclusifs</p>
```

```
        </div>
      </div>
    </div>
  </div>
 );
}
```

8️⃣ COMPOSANT - Confirmation d'Action

```jsx
// src/components/ActionConfirmation.jsx

import { useState } from 'react';
import { useCredits } from '../hooks/useCredits';
import { useRouter } from 'next/router';

export default function ActionConfirmation({
 feature,
 onConfirm,
 onCancel,
 metadata = {}
}) {
 const { credits, getCost, getFeatureName, consume } = useCredits();
 const router = useRouter();
 const [processing, setProcessing] = useState(false);
 const [error, setError] = useState(null);

 const cost = getCost(feature);
 const featureName = getFeatureName(feature);
 const newBalance = credits?.balance - cost;
 const isLowBalance = newBalance < 200;
 const canAfford = credits?.balance >= cost;

 const handleConfirm = async () => {
  try {
   setProcessing(true);
   setError(null);

   const result = await consume(feature, metadata);

   if (result.success) {
    onConfirm?.(result);
   } else if (result.error === 'insufficient_credits') {
    setError('Crédits insuffisants');
```

```
    } else {
      setError('Erreur lors de la consommation des crédits');
    }
  } catch (err) {
    console.error('Erreur confirmation:', err);
    setError(err.message);
  } finally {
    setProcessing(false);
  }
};

return (
  <div className="action-confirmation-overlay">
    <div className="action-confirmation-modal">
      <div className="modal-header">
        <h3>🎯 {featureName}</h3>
        <button
          className="close-btn"
          onClick={onCancel}
          disabled={processing}
        >
          ×
        </button>
      </div>

      <div className="modal-content">
        {/* Coût de l'action */}
        <div className="cost-display">
          <div className="cost-icon">💎</div>
          <div className="cost-details">
            <div className="cost-amount">{cost.toLocaleString()} crédits</div>
            <div className="cost-equivalent">
              ≈ {(cost * 0.25).toFixed(0)} FCFA
            </div>
          </div>
        </div>

        {/* Soldes */}
        <div className="balance-info">
          <div className="balance-row">
            <span className="balance-label">Solde actuel:</span>
            <span className="balance-value current">
              {credits?.balance.toLocaleString()} crédits
            </span>
          </div>
          <div className="balance-row">
            <span className="balance-label">Solde après:</span>
```

```
          <span className={`balance-value after ${isLowBalance ? 'warning' :
''}`}>
            {newBalance.toLocaleString()} crédits
          </span>
        </div>
      </div>

      {/* Avertissement solde bas */}
      {isLowBalance && canAfford && (
        <div className="low-balance-warning">
          <div className="warning-icon">⚠️</div>
          <div className="warning-content">
            <p className="warning-message">
              Attention : Il vous restera moins de 200 crédits
            </p>
            <button
              className="btn-recharge-first"
              onClick={() => router.push('/credits/buy')}
            >
              Recharger d'abord (+10% bonus)
            </button>
          </div>
        </div>
      )}

      {/* Erreur crédits insuffisants */}
      {!canAfford && (
        <div className="insufficient-credits-error">
          <div className="error-icon">❌</div>
          <div className="error-content">
            <p className="error-message">
              Crédits insuffisants
            </p>
            <p className="error-details">
              Il vous manque {(cost - credits?.balance).toLocaleString()} crédits
            </p>
            <button
              className="btn-buy-credits"
              onClick={() => router.push('/credits/buy')}
            >
              💎 Acheter des crédits
            </button>
          </div>
        </div>
      )}

      {/* Erreur générale */}
```

```
        {error && (
          <div className="error-message">
            ⚠️ {error}
          </div>
        )}
      </div>

      <div className="modal-actions">
        {canAfford ? (
          <>
            <button
              className="btn-confirm"
              onClick={handleConfirm}
              disabled={processing}
            >
              {processing ? (
                <>
                  <span className="spinner"></span>
                  Traitement...
                </>
              ) : (
                'Confirmer'
              )}
            </button>
            <button
              className="btn-cancel"
              onClick={onCancel}
              disabled={processing}
            >
              Annuler
            </button>
          </>
        ) : (
          <button
            className="btn-cancel"
            onClick={onCancel}
          >
            Fermer
          </button>
        )}
      </div>
    </div>
  </div>
 );
}
```

9️⃣ COMPOSANT - Historique des Transactions

```jsx
// src/pages/credits/history.jsx

import { useState, useEffect } from 'react';
import { useCredits } from '../../hooks/useCredits';
import CreditWidget from '../../components/CreditWidget';

export default function CreditHistoryPage() {
  const { history, loadHistory, loading, getFeatureName } = useCredits();
  const [filter, setFilter] = useState('all'); // 'all', 'purchase', 'usage'
  const [dateRange, setDateRange] = useState('month'); // 'week', 'month',
'year', 'all'

  useEffect(() => {
    const options = {};

    if (filter !== 'all') {
      options.type = filter;
    }

    // Calculer les dates selon la période
    const now = new Date();
    switch (dateRange) {
      case 'week':
        options.startDate = new Date(now.getTime() - 7 * 24 * 60 * 60 *
1000).toISOString();
        break;
      case 'month':
        options.startDate = new Date(now.getTime() - 30 * 24 * 60 * 60 *
1000).toISOString();
        break;
      case 'year':
        options.startDate = new Date(now.getTime() - 365 * 24 * 60 * 60 *
1000).toISOString();
        break;
    }

    loadHistory(options);
  }, [filter, dateRange]);

  const getTransactionIcon = (type) => {
    switch (type) {
```

```
    case 'purchase': return '💰';
    case 'usage': return '🔺';
    case 'bonus': return '🎁';
    case 'refund': return '↩️';
    default: return '📝';
  }
};

const getTransactionColor = (type) => {
  switch (type) {
    case 'purchase': return '#10B981';
    case 'bonus': return '#F59E0B';
    case 'usage': return '#EF4444';
    case 'refund': return '#3B82F6';
    default: return '#6B7280';
  }
};

const formatDate = (dateString) => {
  const date = new Date(dateString);
  return new Intl.DateTimeFormat('fr-FR', {
    day: '2-digit',
    month: 'short',
    year: 'numeric',
    hour: '2-digit',
    minute: '2-digit',
  }).format(date);
};

if (loading) {
  return <div className="loading">Chargement...</div>;
}

return (
  <div className="history-page">
    {/* Header */}
    <div className="page-header">
      <h1>📊 Historique des Transactions</h1>
      <CreditWidget compact />
    </div>

    {/* Filtres */}
    <div className="filters-bar">
      <div className="filter-group">
        <label>Type:</label>
        <select
          value={filter}
```

```
          onChange={(e) => setFilter(e.target.value)}
          className="filter-select"
        >
          <option value="all">Toutes</option>
          <option value="purchase">Achats</option>
          <option value="usage">Utilisations</option>
          <option value="bonus">Bonus</option>
        </select>
      </div>

      <div className="filter-group">
        <label>Période:</label>
        <select
          value={dateRange}
          onChange={(e) => setDateRange(e.target.value)}
          className="filter-select"
        >
          <option value="week">Cette semaine</option>
          <option value="month">Ce mois</option>
          <option value="year">Cette année</option>
          <option value="all">Tout</option>
        </select>
      </div>
    </div>

    {/* Liste des transactions */}
    <div className="transactions-list">
      {history.length === 0 ? (
        <div className="empty-state">
          <div className="empty-icon">📪</div>
          <h3>Aucune transaction</h3>
          <p>Vos transactions apparaîtront ici</p>
        </div>
      ) : (
        history.map((transaction) => (
          <div
            key={transaction.id}
            className="transaction-item"
            style={{ borderLeftColor: getTransactionColor(transaction.type) }}
          >
            <div className="transaction-icon">
              {getTransactionIcon(transaction.type)}
            </div>

            <div className="transaction-details">
              <div className="transaction-main">
                <span className="transaction-title">
```

```
                {transaction.type === 'purchase' && 'Achat de crédits'}
                {transaction.type === 'usage' &&
getFeatureName(transaction.feature)}
                {transaction.type === 'bonus' && 'Bonus de fidélité'}
                {transaction.type === 'refund' && 'Remboursement'}
              </span>
              <span
                className={`transaction-amount ${transaction.amount > 0 ?
'positive' : 'negative'}`}
              >
                {transaction.amount > 0 ? '+' : ''}
{transaction.amount.toLocaleString()}
              </span>
            </div>

            <div className="transaction-meta">
              <span className="transaction-date">
                {formatDate(transaction.created_at)}
              </span>
              <span className="transaction-balance">
                Solde: {transaction.balance_after.toLocaleString()}
              </span>
            </div>

            {/* Détails additionnels */}
            {transaction.package_id && (
              <div className="transaction-extra">
                Package: {transaction.package_id}
              </div>
            )}
            {transaction.payment_method && (
              <div className="transaction-extra">
                Via: {transaction.payment_method}
              </div>
            )}
          </div>
        </div>
      ))
    )}
  </div>

  {/* Statistiques */}
  {history.length > 0 && (
    <div className="history-stats">
      <h3>📈 Statistiques de la période</h3>
      <div className="stats-grid">
        <div className="stat-card">
```

```
            <div className="stat-label">Total acheté</div>
            <div className="stat-value positive">
              +{history
                .filter(t => t.type === 'purchase' || t.type === 'bonus')
                .reduce((sum, t) => sum + t.amount, 0)
                .toLocaleString()}
            </div>
          </div>
          <div className="stat-card">
            <div className="stat-label">Total utilisé</div>
            <div className="stat-value negative">
              {history
                .filter(t => t.type === 'usage')
                .reduce((sum, t) => sum + t.amount, 0)
                .toLocaleString()}
            </div>
          </div>
          <div className="stat-card">
            <div className="stat-label">Transactions</div>
            <div className="stat-value">
              {history.length}
            </div>
          </div>
        </div>
      </div>
    )}
  </div>
 );
}
```

🔟 API ROUTES

```javascript
// src/pages/api/credits/balance.js

import { supabase } from '../../../lib/supabase';
import { getUserBalance } from '../../../lib/credits';

export default async function handler(req, res) {
 if (req.method !== 'GET') {
  return res.status(405).json({ error: 'Method not allowed' });
 }

 try {
```

```
    // Récupérer l'utilisateur authentifié
    const token = req.headers.authorization?.replace('Bearer ', '');
    const { data: { user }, error: authError } = await
supabase.auth.getUser(token);

    if (authError || !user) {
      return res.status(401).json({ error: 'Unauthorized' });
    }

    // Récupérer le solde
    const balance = await getUserBalance(user.id);

    return res.status(200).json({
      success: true,
      data: balance,
    });
  } catch (error) {
    console.error('Erreur API balance:', error);
    return res.status(500).json({
      success: false,
      error: error.message,
    });
  }
}
```

```javascript
// src/pages/api/credits/purchase.js

import { supabase } from '../../../lib/supabase';
import { initiateMobileMoneyPayment } from '../../../lib/momo-payment';

export default async function handler(req, res) {
  if (req.method !== 'POST') {
    return res.status(405).json({ error: 'Method not allowed' });
  }

  try {
    // Auth
    const token = req.headers.authorization?.replace('Bearer ', '');
    const { data: { user }, error: authError } = await
supabase.auth.getUser(token);

    if (authError || !user) {
      return res.status(401).json({ error: 'Unauthorized' });
    }
```

```
    const { packageId, phoneNumber, operator } = req.body;

    // Validation
    if (!packageId || !phoneNumber || !operator) {
      return res.status(400).json({
        error: 'Missing required fields'
      });
    }

    // Récupérer le package
    const { data: pkg, error: pkgError } = await supabase
      .from('credit_packages')
      .select('*')
      .eq('id', packageId)
      .single();

    if (pkgError || !pkg) {
      return res.status(404).json({ error: 'Package not found' });
    }

    // Initier le paiement
    const result = await initiateMobileMoneyPayment({
      userId: user.id,
      packageId,
      amount: pkg.price_fcfa,
      phoneNumber,
      operator,
    });

    return res.status(200).json({
      success: true,
      data: result,
    });
  } catch (error) {
    console.error('Erreur API purchase:', error);
    return res.status(500).json({
      success: false,
      error: error.message,
    });
  }
}
```

```javascript
// src/pages/api/credits/consume.js
```

```
import { supabase } from '../../../lib/supabase';
import { consumeCredits } from '../../../lib/credits';

export default async function handler(req, res) {
  if (req.method !== 'POST') {
    return res.status(405).json({ error: 'Method not allowed' });
  }

  try {
    // Auth
    const token = req.headers.authorization?.replace('Bearer ', '');
    const { data: { user }, error: authError } = await
supabase.auth.getUser(token);

    if (authError || !user) {
      return res.status(401).json({ error: 'Unauthorized' });
    }

    const { feature, metadata } = req.body;

    if (!feature) {
      return res.status(400).json({ error: 'Missing feature' });
    }

    // Consommer les crédits
    const result = await consumeCredits(user.id, feature, metadata);

    return res.status(200).json({
      success: true,
      data: result,
    });
  } catch (error) {
    console.error('Erreur API consume:', error);

    if (error.code === 'insufficient_credits') {
      return res.status(402).json({
        success: false,
        error: 'insufficient_credits',
        details: error.details,
      });
    }

    return res.status(500).json({
      success: false,
      error: error.message,
    });
```

```
  }
}
```

```javascript
// src/pages/api/credits/webhook.js

import { handlePaymentWebhook } from '../../../lib/momo-payment';

export default async function handler(req, res) {
  if (req.method !== 'POST') {
    return res.status(405).json({ error: 'Method not allowed' });
  }

  try {
    // TODO: Valider la signature du webhook selon l'opérateur

    const result = await handlePaymentWebhook(req.body);

    return res.status(200).json({
      success: true,
      data: result,
    });
  } catch (error) {
    console.error('Erreur webhook:', error);
    return res.status(500).json({
      success: false,
      error: error.message,
    });
  }
}
```